Phase 5 | Project Development Phase

Coding & Solution

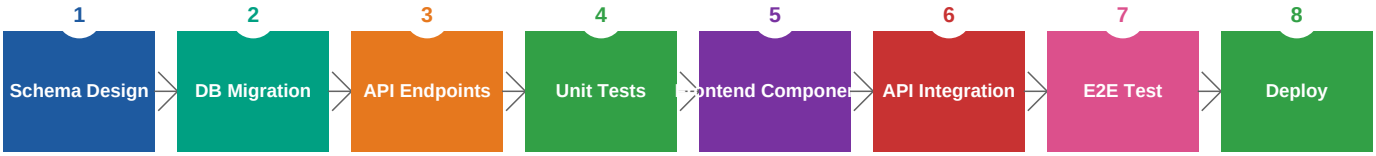
AI-Powered Travel Planner | Smart Situational Itinerary Engine

Live URL:	https://travel-planner-iota-plum.vercel.
GitHub:	github.com/itskian2007/travel-planner
Tech:	React + FastAPI + PostgreSQL + Vercel
Team:	itskian2007

1. Development Approach

Development followed an API-first approach: backend endpoints were built and documented via FastAPI's auto-generated Swagger UI before frontend integration. React components were developed in isolation using prop-driven design, then wired to real API data.

2. Development Workflow Diagram



3. Key Code Implementations

FastAPI - Itinerary Generation Endpoint

```
@router.post("/generate", response_model=ItineraryResponse)
async def generate_itinerary(
    prefs: ItineraryPreferences,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user)
):
    """Generate a personalised 10-day itinerary."""
    destination = get_destination(db, prefs.destination_id)
    curated_days = get_curated_days(destination, days=3)
    procedural_days = generate_procedural_days(
        destination, prefs, start_day=4, end_day=10
    )
    return ItineraryResponse(
        days=curated_days + procedural_days,
        destination=destination.name
    )
```

```
const FilterPanel = ({ onFilterChange }) => {
```

```
const [filters, setFilters] = useState({
```

```
mood: "", budget: "", diet: "",
```

```
persona: "", accessibility: false
```

```
});
```

```
const handleChange = (key, value) => {
```

```
const updated = { ...filters, [key]: value };
```

```
setFilters(updated);
```

```
onFilterChange(updated);  // lift state to parent
```

```
};
```

```
return (
```

```
<div className="filter-panel bg-white rounded-xl p-6">
```

```
<MoodSelector value={filters.mood}
```

```
onChange={(v) => handleChange("mood", v)} />
```

```
<BudgetSelector value={filters.budget}
```

```
onChange={(v) => handleChange("budget", v)} />
```

```
</div>
```

```
);
```

```
};
```

Procedural Day Engine - Core Logic

```
def generate_procedural_days(dest, prefs, start_day, end_day):
    """Generate Days 4-10 based on destination and preferences."""
    activity_pool = dest.activities[prefs.mood]
    dining_pool   = filter_by_diet(dest.restaurants, prefs.diet)
    budget_factor = BUDGET_MULTIPLIERS[prefs.budget]

    days = []
    used_activities = set()
    for day_num in range(start_day, end_day + 1):
        available = [a for a in activity_pool
                     if a.id not in used_activities]
        selected = random.sample(available, min(3, len(available)))
        used_activities.update(a.id for a in selected)
        days.append(DayPlan(
            day=day_num,
            attractions=selected,
            meals=random.sample(dining_pool, 2),
            budget_estimate=sum(a.cost for a in selected) * budget_factor
        ))
    return days
```